NewDOS/80 Disk Basic Detokenizer Design Document

Sharky J Yellowfish
Screaming Yellow Fish Engineering
March 14, 2026

Project: NewDos80BasicDetokenizer
Target dialect: TRS-80 Model III/4 NewDOS/80 Disk BASIC
Document status: Working design baseline

Purpose	Describe the current file model, detokenization rules, architecture, error-handling approach, and current limitations.
Output target	Readable ASCII BASIC source suitable for inspection, diffing, archival use, and regression testing.
Version note	Current project milestone discussed in this document: 0.8.0.

1. Overview

NewDos80BasicDetokenizer is a C# console utility that converts tokenized TRS-80 Model III/4 NewDOS/80 Disk BASIC program files into readable ASCII BASIC source.

The tool is intended for source recovery, archival preservation, regression testing, and analysis of historical BASIC programs originally saved in tokenized Disk BASIC format.

The design goal is to reconstruct source as faithfully as practical while still surfacing unresolved bytes and format ambiguities clearly.

2. Goals

- Parse tokenized NewDOS/80 Disk BASIC program files correctly.
- Reconstruct linked BASIC program lines into readable ASCII source.
- Preserve program meaning over cosmetic formatting.
- Preserve raw user-authored text where token decoding should stop.
- Surface unresolved bytes visibly rather than silently corrupt output.
- Support repeatable regression testing with known input/output pairs.
- Remain simple enough to maintain and extend as more token behavior is learned.

3. Non-Goals

- Full BASIC syntax validation.
- Automatic correction of source logic.
- Enforcing modern pretty-print formatting.
- Re-tokenizing ASCII back into NewDOS/80 binary form.

- Perfect reconstruction of every original spacing choice in all cases.
- Emulation of the TRS-80 BASIC runtime.

4. Scope

This tool currently focuses on tokenized program input files saved by NewDOS/80 Disk BASIC, linked-line program record parsing, keyword, function, and operator token expansion, quoted string preservation, raw comment preservation, and readable ASCII output generation.

The tool does not currently attempt to decode every possible dialect extension without evidence, execute BASIC, or infer intent beyond deterministic parsing rules.

5. Input File Model

The tokenized BASIC file format is treated as follows:

```
0xFF
repeat
    uint16 nextLinePtr   (little endian)
    uint16 lineNumber    (little endian)
    byte[] tokenized line body
    0x00                 end-of-line
until nextLinePtr == 0 && lineNumber == 0
```

The file begins with a leading 0xFF marker.

Each BASIC line is stored as a linked record.

The parser must stop when both nextLinePtr and lineNumber are zero. This prevents trailing padding from being misread as a fake BASIC line.

6. Output Model

The output is ASCII text consisting of one reconstructed BASIC source line per logical BASIC line record.

Each output line is emitted as:

```
<line-number><space><detokenized-line-text>
```

If a line has no body text, only the line number is emitted.

Output line termination is generated by the detokenizer, not copied from embedded bytes inside tokenized line content.

7. High-Level Architecture

The utility is logically split into three responsibilities:

7.1 Program-level parsing

Reads the file byte stream, skips the file marker, walks linked BASIC line records, extracts each tokenized line body, and passes it to line detokenization.

7.2 Line-level detokenization

Processes one tokenized line body left to right, expanding tokens, preserving ASCII, handling strings, handling comments, and normalizing output.

7.3 CLI shell

Provides console banner, argument validation, usage and help output, file I/O, and error reporting.

8. Core Processing Flow

8.1 Program flow

1. Validate the input byte array.
2. Skip a leading 0xFF if present.
3. Loop while at least four bytes remain for a line header.
4. Read nextLinePtr and lineNumber.
5. Stop if both values are zero.
6. Find the 0x00 line terminator.
7. Extract the tokenized line body.
8. Detokenize the line body.
9. Emit the line number, optional space plus detokenized text, and a newline.
10. Advance to the next line record.
11. Stop if nextLinePtr is zero.

8.2 Line flow

Each line is processed left to right with a small set of lexical states: normal mode, inside quoted string, and raw-text mode for comments.

The line detokenizer decides, byte by byte, whether to copy visible ASCII, expand a token, fold a compound operator, enter or exit quote mode, or stop tokenizing and copy the rest as raw ASCII.

9. Detokenization States

9.1 Normal mode

Default processing mode. In this mode, the detokenizer may copy ASCII bytes below 0x80, recognize keyword, function, and operator tokens at or above 0x80, detect compound operators, detect quote start, and detect comment start.

9.2 Quote mode

Entered when a double quote is encountered. While in quote mode, bytes are copied literally as visible ASCII, token expansion is disabled, and the mode exits at the next double quote.

9.3 Raw comment mode

Used when the remainder of the line must no longer be tokenized. It is triggered by REM and apostrophe-comment forms. Once entered, the rest of the line is treated as ASCII text, token expansion stops, and spacing normalization is bypassed for the raw remainder.

10. Token Handling

10.1 Token map

Bytes at or above 0x80 are mapped through a token table to BASIC keywords, functions, and operators.

Examples include 0x8F -> IF, 0xAF -> LPRINT, 0xB1 -> POKE, 0xC5 -> INSTR, 0xC7 -> TIME$, 0xDA -> FRE, and 0xF3 -> LEN.

Later analysis of real program output led to inferred mappings such as 0xBF -> USING, 0xC0 -> VARPTR, and 0xC1 -> USR.

10.2 Unknown tokens

If a byte at or above 0x80 is not recognized, it is emitted visibly as:

```
<$XX>
```

This is intentional. Unknown bytes should remain obvious so the token table can be refined safely instead of silently generating incorrect source.

11. Compound Operator Folding

Some relational operators are encoded as adjacent token pairs rather than as a single logical output string.

The detokenizer folds the following token sequences:

- `< + >` -> `<>`
- `< + =` -> `<=`
- `> + =` -> `>=`

This avoids broken output such as:

```
IF FK < >1
```

and emits:

```
IF FK<>1
```

A look-ahead helper performs this folding before ordinary single-token emission.

12. String and Comment Preservation

12.1 Strings

Quoted strings are copied literally from opening " to closing ". This preserves punctuation, spaces, digits, and other visible ASCII without accidental token expansion.

12.2 REM comments

When REM is encountered, REM is emitted, the remainder of the line is copied as raw ASCII, and no further token processing occurs on that line.

12.3 Apostrophe comments

Apostrophe comments are handled in whole-line stored special form, inline form after a REM plus apostrophe token sequence, and direct apostrophe-token cases. In all such cases, the remainder of the line becomes raw text.

13. Visible ASCII Handling

Visible 7-bit ASCII bytes are copied directly.

Special treatment currently includes:

- 0x0A ignored when embedded inside tokenized line data
- 0x0D ignored when embedded inside tokenized line data
- printable 0x20 through 0x7E copied directly
- non-printable or otherwise unrecognized bytes shown as \xXX

Ignoring embedded 0x0A and 0x0D prevents accidental splitting of a single logical BASIC line into multiple output lines.

14. Spacing Strategy

Spacing is one of the hardest fidelity issues in source reconstruction.

The current strategy is conservative:

- avoid duplicate spaces
- avoid synthetic spaces before function-call parentheses
- avoid operator corruption
- normalize obvious spacing artifacts
- preserve raw comment text without normalization

Current spacing behavior includes:

- no synthetic space before (for function-style forms such as LEN(X$), INSTR(A$,"%"), and VARPTR(ID$)
- operator-aware handling for +, -, *, /, ^, =, >, <, <>, <=, and >=

- reduced duplicate or awkward spacing

Known spacing limitation: output is improved but not yet guaranteed to match original TRS-80 ASCII SAVE formatting byte for byte in all cases.

15. Error-Handling Philosophy

The tool favors deterministic parsing, visible failures, and non-destructive output.

Current error strategy includes:

- empty or null input raises an exception
- unresolved tokens remain visible
- malformed control bytes are surfaced rather than hidden
- program-level parsing stops safely at the established end condition

This is preferable to guessing silently and producing plausible but incorrect source.

16. CLI Design

The utility is a console application.

Banner

At startup it prints a one-line banner such as:

```
NewDOS/80 Disk Basic Detokenizer Utility v0.8.0
```

Help and usage

Expected arguments:

```
ND80BasicDetokenizar.exe <input.bas> <output.asc>
```

Supported help switches:

- -h
- --help
- /h
- /?

The CLI is intentionally small and direct.

17. Testing Strategy

Regression testing is based on known tokenized and ASCII file pairs.

Suggested project layout:

```
NewDos80BasicDetokenizer/
    Tests/
        Data/
            install.bas
            install.asc
            mainpost.bas
            mainpost.asc
```

Test pair meaning

- .bas = tokenized input
- .asc = expected detokenized output

Current testing approach

- run the detokenizer on known tokenized input
- compare generated output against the expected ASCII file
- inspect visible token markers or spacing differences
- refine the token map and formatting rules incrementally

This approach supports safe iteration on real-world historical programs.

18. Known Issues and Open Items

- some USR forms still have spacing artifacts
- formatting is improved but not yet fully canonical versus original ASCII save output
- additional token meanings may still need confirmation from real source files
- future refinements may reduce normalization side effects further

Important intentional behavior

Missing THEN after IF is not automatically treated as an error. This is expected in some source styles and should be preserved when encountered.

19. Future Work

- refine spacing around USR forms and similar edge cases
- further reduce synthetic formatting artifacts
- expand token coverage from additional sample programs
- add automated regression test execution
- consider a diff-friendly test harness
- optionally add verbose or debug dump modes for token analysis
- optionally support token table diagnostics or trace output

Longer-term possibilities include a token re-encoder, deeper dialect analysis, and structured parse output for tooling.

20. Design Principles

20.1 Preserve meaning first

Semantic correctness matters more than cosmetic formatting.

20.2 Never hide uncertainty

Unknown bytes should remain visible until proven.

20.3 Use evidence from real files

Token assignments should be driven by actual program usage, not guesswork alone.

20.4 Keep the implementation understandable

The code should remain straightforward enough to audit and extend without turning into a full compiler.

21. Summary

- linked BASIC program structure
- token expansion
- strings
- comments
- compound relational operators
- visible unknown-token output
- basic CLI usability

The current implementation is useful and increasingly accurate, while still leaving room for further refinement in formatting fidelity and dialect-specific token behavior.